

实验细节

本次实验包含 3 个任务：

1. 迁移上一章的 `sys_get_time` , `sys_mmap` , `sys_munmap` 。
2. 实现 `sys_spawn` 系统调用。
3. 实现 `stride` 调度算法。

对于上一章节的 `sys_get_time` , 系统调用, 我原先是使用了一个 `copy_to_user` 的函数进行实现, 这次反应过来, 直接采用了已经提供的 `translated_byte_buffer` 函数获取对应页的物理地址, 然后进行逐字节修改数据。 `sys_mmap` , `sys_munmap` 的实现与上一章节类似, 主要处理了异常从哪里开始传播的问题。

在实现 `sys_spawn` 系统调用中, 我主要参考了 `fork` 和 `exec` 的实现, 也就是说, 首先从 elf 文件中获取到进程的初始状态, 然后基于该状态构建一个新的进程, 并输入父进程的基本信息即可。

在 `stride` 调度算法中, 其中的细节分为两部分：

1. `TaskControlBlock` 的增强：我向 `TaskControlBlockInner` 添加了 `stride` , `priority` 两个属性来维护优先级状态和步长状态。每次调度时 `stride` 增加 `BIG_STRIDE / self.priority` 的大小, 其中 `BIG_STRIDE` 是一个常量, 我设置为 `usize::MAX >> 1` 。
2. `StrideTaskManager` 的实现：我写了一个 `StrideTaskManager` 以顶替原先的 `TaskManager` 用作调度器。我给 `TaskControlBlock` 实现了 `PartialEq` , `Eq` , `PartialOrd` , `Ord` 四个 trait, 这样我可以使用核心库中的 `BinaryHeap` 维护一个小顶堆, 以此实现 `stride` 调度算法。

问答题

问题一

答：不是, 因为此时 `p2.stride = 250` 将会溢出为 5, 因此还是 `p2` 最小, 依旧是 `p2` 优先执行。

问题二

答：因优先级 ≥ 2 , 因此步长 `pass` 最大值为 `BigStride / 2`, 考虑到每次调度最小 `stride` 的进程, 因此 `STRIDE_MAX - STRIDE_MIN` 将小于最大步长, 即 `STRIDE_MAX - STRIDE_MIN <= BigStride / 2`。

问题三

答：代码如下, 由于 `BinaryHeap` 的实现是一个大顶堆, 所以我在实现 `PartialOrd` 时, 返回的顺序是相反的。

```
1 use core::cmp::Ordering;
2
```

rust

```

3  struct Stride(u64);
4
5  impl PartialOrd for Stride {
6      fn partial_cmp(&self, other: &Self) -> Option<Ordering> {
7          match self.0.cmp(&other.0) {
8              Ordering::Less => Some(Ordering::Greater),
9              Ordering::Greater => Some(Ordering::Less),
10             _ => None,
11         }
12     }
13 }
14
15 impl PartialEq for Stride {
16     fn eq(&self, other: &Self) -> bool {
17         false
18     }
19 }

```

荣誉准则

1. 在完成本次实验的过程（含此前学习的过程）中，我曾分别与 以下各位 就（与本次实验相关的）以下方面做过交流，还在代码中对应的位置以注释形式记录了具体的交流对象及内容：
 - 我们伟大的群友们
2. 此外，我也参考了 以下资料，还在代码中对应的位置以注释形式记录了具体的参考来源及内容：
 - 《RISC-V 开放架构设计之道 1.0.0 (原著 The RISC-V Reader: An Open Architecture Atlas)》
 - 《RISC-V Supervisor Binary Interface Specification》
 - KiMi, Deepseek
 - 《Linux 内核深度解析》
 - 《The RISC-V Instruction Set Manual: Volume II》
3. 我独立完成了本次实验除以上方面之外的所有工作，包括代码与文档。我清楚地知道，从以上方面获得的信息在一定程度上降低了实验难度，可能会影响起评分。
4. 我从未使用过他人的代码，不管是原封不动地复制，还是经过了某些等价转换。我未曾也不会向他人（含此后各届同学）复制或公开我的实验代码，我有义务妥善保管好它们。我提交至本实验的评测系统的代码，均无意于破坏或妨碍任何计算机系统的正常运转。我清楚地知道，以上情况均为本课程纪律所禁止，若违反，对应的实验成绩将按“-100”分计。